When Selection Gives Join

Step 4 of *1000 Steps Toward Session Types as Algebraic Reticulates*

Alexandre Zua Caldeira
Independent Researcher
zua@bica-tools.org

March 2026

Abstract

In Step 3 we proved that the branch constructor creates meets: disjoint sub-state-spaces force every cross-branch meet to be $\bot$. In this paper—Step 4—we complete the picture by showing that the selection constructor $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ creates joins. The key insight is that branch and selection produce *structurally identical state spaces*: the graph is the same; only the polarity metadata differs. Therefore the Choice Lemma from Step 3 applies immediately. The selection entry state q_0 is the join of all sub-entry states, and the cross-branch join is $\top$. While the formal result is a one-line corollary, the *interpretation* is conceptually distinct: where branching models external choice (the environment decides), selection models internal choice (the object decides). We explore this duality through worked examples from the Java Iterator, SMTP, OAuth 2.0, and ATM benchmarks, and connect selection's role to Gay–Hole subtyping, where selection is contravariant (fewer labels = subtype). The result validates on all 79 binary benchmarks.

1 Introduction

1.1 Recap: Where We Stand

After Step 3, we know that the branch constructor preserves lattice structure. The proof rests on a single structural property: branch creates *disjoint sub-state-spaces* sharing only $\bot$, so cross-branch meets are trivially $\bot$ and cross-branch joins are trivially $\top$. The argument is captured in the Choice Lemma, which shows that the fan-shaped arrangement of sub-protocols creates all meets and joins required by the lattice axioms.

But Step 3 focused on the branch constructor $\&\{m_1 : S_1, \ldots, m_n : S_n\}$ and its role in creating meets. What about selection $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$? Does it also preserve lattice structure? And if so, what is its specific contribution to the lattice—does it create meets, joins, or both?

1.2 The Punchline

The answer is surprisingly simple: **selection produces the exact same graph as branch**. The state-space construction (Step 1, Rules 2 and 3) treats branch and selection identically: both create a fresh entry state q_0 with transitions to each sub-state-space. The only difference is a metadata annotation: for selection, the transitions $q_0 \xrightarrow{l_i} q_\top^{S_i}$ are marked in the `selection_transitions` set, recording that these are *internal choices* rather than *external choices*.

Since the graph is identical, the Choice Lemma from Step 3 applies immediately. Selection preserves lattice structure for exactly the same reason branch does: disjoint sub-state-spaces, shared $\bot$, fan-shaped graph. The formal result is a one-line corollary.

So why does Step 4 get its own paper? Because the *interpretation* is different. Where branch models the environment choosing which method to call (external choice), selection models the object choosing which outcome to signal (internal choice). The graph may be the same, but the operational meaning is dual. Understanding this duality is essential for the subtyping theory (Step 7) and the duality theory (Step 8), where branch and selection play opposite roles.

This duality has deep roots. In the Curry–Howard correspondence between session types and linear logic [1, 5], branch corresponds to the additive conjunction & ("with") and selection to the additive disjunction $\oplus$ ("plus"). These are De Morgan duals in linear logic: $\overline{A \& B} = \overline{A} \oplus \overline{B}$. Our observation that they produce the same state space is the lattice-theoretic reflection of this logical duality: De Morgan duals have the same denotational structure, differing only in polarity.

1.3 What This Paper Covers

Section 2 establishes the structural identity between branch and selection at the state-space level. Section 3 states the formal corollary. Section 4 explores the interpretive difference: selection as internal choice, and why it gives "join" rather than "meet" in the operational reading. Section 5 walks through four worked examples. Section 6 connects selection to Gay–Hole subtyping. Section 7 previews the branch/selection duality. Section 8 validates on all 79 benchmarks. Section 9 discusses the pedagogical and theoretical implications, and Section 11 concludes.

2 The Structural Identity

2.1 Branch and Selection: Same Graph, Different Metadata

Let us place the two constructions side by side. Given $S_\& = \&\{m_1 : S_1, \ldots, m_n : S_n\}$ and $S_\oplus = \oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ with the *same* continuations $S_1, \ldots, S_n$:

	Branch $\&\{\ldots\}$	Selection $\oplus\{\ldots\}$
Entry state	Fresh q_0	Fresh q_0
Transitions	$q_0 \xrightarrow{m_i} q_\top^{S_i}$	$q_0 \xrightarrow{l_i} q_\top^{S_i}$
Sub-state-spaces	Disjoint above $q_\bot$	Disjoint above $q_\bot$
Terminal state	Shared $q_\bot$	Shared $q_\bot$
`selection_transitions`	$\emptyset$	$\{(q_0, l_i, q_\top^{S_i})\}$

The graphs are identical. The only difference is the last row: for selection, the transitions from q_0 are flagged as "internal choices." This flag is consumed by the subtyping checker (Step 7) and the duality checker (Step 8), but it has *no effect on the reachability structure*, which is what determines the lattice.

Figure 1 illustrates this visually. The two diamonds are the same graph; only the coloring differs.

2.2 Why the Identity Holds

The structural identity is a consequence of how we define state spaces. A state space is a labeled transition system capturing *which actions are possible from each state*. Both branch and selection offer the same set of actions from the entry state: $m_1, \ldots, m_n$ (or $l_1, \ldots, l_n$). The difference is *who chooses*: in branch, the environment picks; in selection, the object picks. But "who chooses" is not a graph property—it is a *polarity* annotation.

This design choice is deliberate. We could have introduced separate graph constructions for branch and selection (e.g., different node shapes, different edge types). But that would complicate the lattice proof without adding information: the lattice property depends only on reachability, and reachability is polarity-blind. By keeping the graphs identical and the polarity in metadata, we get two constructor lemmas for the price of one.

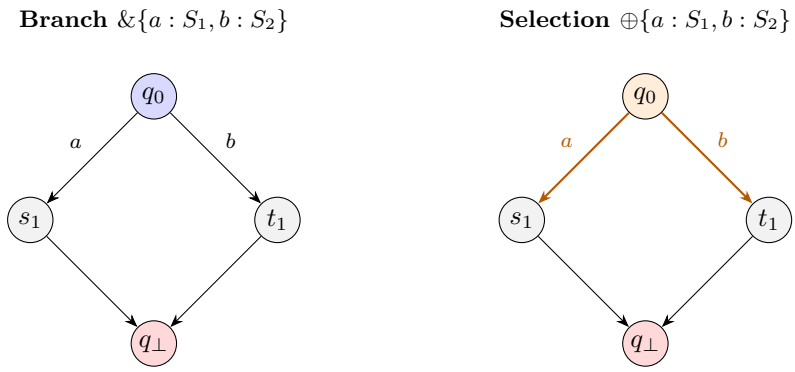

Figure 1: Branch and selection with the same continuations produce identical graphs. The orange entry state and thick arrows mark selection transitions (internal choice); the graph structure is the same diamond M_2 in both cases.

2.3 The Unified "Choice" Constructor

In the formal development, we therefore define a single "choice" constructor $C\{a_1 : S_1, \ldots, a_n : S_n\}$ that covers both branch and selection. The constructor C is either & or $\oplus$, but the state-space builder ignores this distinction. The Choice Lemma (Step 3) proves that $C\{\ldots\}$ preserves lattice structure regardless of the polarity.

This unification simplifies the Reticulate Theorem's structural induction: instead of six constructor cases (end, sequencing, branch, selection, recursion, parallel), we have five (end, sequencing, choice, recursion, parallel). Branch and selection are a single case.

3 The Corollary

Corollary 3.1 (Selection preserves lattice structure)**.** *If $\mathcal{L}(S_i)/\equiv$ is a bounded lattice for each $i = 1, \ldots, n$, then $\mathcal{L}(\oplus\{l_1 : S_1, \ldots, l_n : S_n\})/\equiv$ is a bounded lattice.*

Proof. The state-space construction for $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ produces the same graph as for $\&\{l_1 : S_1, \ldots, l_n : S_n\}$. Apply the Choice Lemma (Step 3, Lemma 3.1). □

That is the entire formal content of Step 4: a one-line appeal to Step 3. The remainder of this paper is devoted to *understanding* this result—why it matters, what it means operationally, and how it interacts with subtyping and duality.

4 The Interpretive Difference

4.1 External vs. Internal Choice

Although the graphs are identical, the operational meanings of branch and selection are profoundly different.

Branch (external choice, &). The *environment* (the caller, the client) chooses which method to invoke. The object must be prepared for any of the n options. In type-theoretic terms, a branch type is a *product*: the object offers m_1 *and* m_2 *and* $\ldots$ *and* m_n. A broader branch (more methods) is a *subtype*: it offers more, so it can be used wherever a narrower branch is expected.

Selection (internal choice, $\oplus$). The *object* (the server, the callee) chooses which outcome to signal. The environment must be prepared to handle any of the n possibilities. In type-theoretic terms, a selection type is a *coproduct*: the object produces l_1 *or* l_2 *or* $\ldots$ *or* l_n. A

narrower selection (fewer labels) is the *subtype*: it signals fewer possibilities, so it is easier for the environment to handle.

4.2 Why Selection "Gives Join"

In the state-space lattice, the entry state q_0 of a selection is the *join* of the sub-entry states $q_\top^{S_1}, \ldots, q_\top^{S_n}$. This is because q_0 is the least state from which all sub-protocols are reachable: choosing any label l_i transitions to $q_\top^{S_i}$, and no state below q_0 can reach all of them (the sub-state-spaces are disjoint).

Operationally: at state q_0, the protocol's future is a *join of possibilities*. The object will select one label, narrowing the future to a single sub-protocol. Before the selection, all futures are possible; after it, exactly one is chosen. The entry state represents the *superposition of all possible outcomes*, which is exactly the join in the information ordering.

Dually, in Step 3 we saw that the branch entry state is also the join of the sub-entry states (from the lattice perspective, $q_0 = \top$ is above all sub-entries). So both branch and selection entry states are joins in the *state-space lattice*. The difference is in the *subtyping lattice*: branch with more methods is a subtype (lower in the subtyping order), while selection with fewer labels is a subtype. We develop this in Section 6.

4.3 The Join Table

Just as Step 3 showed that cross-branch meets are always $\bot$, we can observe that cross-branch joins are always $\top$. For any $[s] \in L_i$ and $[t] \in L_j$ with $i \neq j$:

$$[s] \sqcap [t] = [q_\bot] = \bot \qquad \text{(Step 3, Case 2)}$$
$$[s] \sqcup [t] = [q_0] = \top \qquad \text{(symmetric argument)}$$

The join argument mirrors the meet argument exactly: a common upper bound of $[s]$ and $[t]$ must reach both. Since $[s]$ is reachable only from states in L_i or from $[q_0]$, and similarly for $[t]$, the only common ancestor is $[q_0]$. Therefore $[s] \sqcup [t] = [q_0] = \top$.

This symmetry is the formal expression of the branch/selection duality at the state-space level. Both constructors create the same graph; both contribute both meets *and* joins. The "branch gives meet, selection gives join" slogan is an interpretive convenience, not a formal asymmetry.

5 Worked Examples

5.1 Java Iterator: The Simplest Selection

The Java Iterator protocol is the simplest benchmark featuring selection:

$$S_{iter} = \mu X.\, \&\{hasNext : \oplus\{TRUE : \&\{next : X\},\ FALSE : \mathbf{end}\}\}$$

After querying `hasNext`, the server *selects* between `TRUE` (the iterator has more elements) and `FALSE` (the iterator is exhausted). The client cannot choose—it must accept whichever outcome the server signals.

The selection creates a diamond: `sel` is the join of `next` and $\bot$. The entry state represents the superposition "the iterator might have more elements, or it might not." Once the selection fires, the future is determined.

This is the prototypical use of selection: a *server-driven decision point* where the object (the iterator) decides its own state and communicates the result to the client.

It is worth contrasting this with how a *branch* version would look. If the iterator used branch instead of selection—say $\&\{next : X,\ done : \mathbf{end}\}$—the *client* would decide whether to

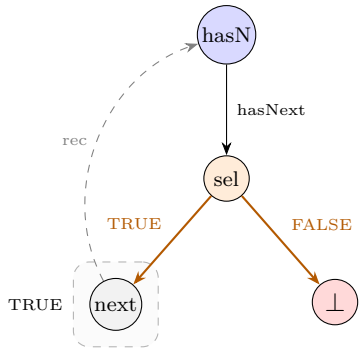

Figure 2: Java Iterator: the selection node (orange) internally chooses between TRUE and FALSE. The TRUE branch loops back via recursion; the FALSE branch terminates. After SCC quotient, the recursive states collapse.

call `next` or `done`. The graph would be the same diamond, but the operational meaning would be different: the client controls the iteration, calling `next` until it decides to stop.

In the real Java Iterator, the server controls termination (the collection knows when it is exhausted), so selection is the correct polarity. The state-space lattice is the same in both cases—but the subtyping and test generation differ.

5.2 SMTP: Selection as Server Response

The SMTP protocol features selection after the `data` command:

$$\ldots \&\{data : \oplus\{OK : X,\ ERR : X\}\} \ldots$$

After the client sends data, the *server* selects between `OK` (data accepted, loop back for more mail) and `ERR` (data rejected, loop back to try again). The client cannot influence this decision—it must handle both outcomes.

Here both branches loop back to the same recursive variable X, so after the SCC quotient, the selection's sub-entry states collapse into the same equivalence class. The selection node and both outcomes merge into a single SCC. This is a degenerate case: the selection exists in the AST but is invisible in the quotient DAG because recursion absorbs it.

The lesson: *selection can be masked by recursion.* At the AST level, the object genuinely makes a choice (OK vs. ERR). At the state-space level, both choices lead to the same protocol state (the loop entry), so the lattice does not distinguish them. The selection metadata is preserved for subtyping and test generation, but the lattice structure is unaffected.

This is worth dwelling on because it reveals a subtle distinction between the AST and the state space. The AST is a *syntactic* object: it records every constructor, including selections whose branches happen to converge. The state space is a *semantic* object: it records only the reachable states and transitions, and convergent branches produce the same successor state. The Choice Lemma operates on the state space, so it "sees" the convergence and produces a simpler lattice. But the AST-level selection is still real: a test generator must produce code that handles both `OK` and `ERR` outcomes, even though the protocol proceeds identically in both cases.

This AST/state-space gap is a recurring theme in session type theory. The syntax carries information (polarity, selection metadata, variable names) that the semantics erases. The lattice theorem operates at the semantic level, which is why it is polarity-blind. But applications like test generation, subtyping, and conformance checking operate at the syntactic level and use the erased information.

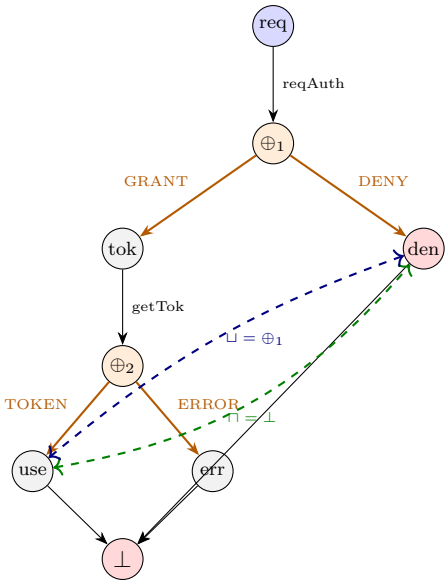

Figure 3: OAuth 2.0 (simplified, omitting recursion and the refresh selection). Orange nodes are selection points. The meet of any GRANTED-branch state with the DENIED state is $\bot$; their join is $\oplus_1$ (the outermost selection entry).

5.3 OAuth 2.0: Nested Selections

OAuth 2.0 is the benchmark with the most nested selections:

$$S_{oauth} = \&\{reqAuth : \oplus\{GRANTED : \&\{getToken : \oplus\{TOKEN : S_{use},\ ERROR : \mathbf{end}\}\},\ DENIED : \mathbf{end}\}\}$$

where $S_{use} = \mu X.\&\{useToken : X,\ refreshToken : \oplus\{OK : X,\ EXPIRED : \mathbf{end}\},\ revoke : \mathbf{end}\}$.

There are *three* selection points:

1. $\oplus\{GRANTED, DENIED\}$ — the authorization server decides whether to grant access.
2. $\oplus\{TOKEN, ERROR\}$ — the token endpoint decides whether to issue a token.
3. $\oplus\{OK, EXPIRED\}$ — the refresh endpoint decides whether the refresh succeeds.

Each selection creates a fan of disjoint sub-state-spaces. The outermost selection (GRANTED/DENIED) creates two branches: the GRANTED branch is a rich sub-lattice (with the token lifecycle inside), while the DENIED branch is a single-state path to $\bot$.

The meet of any state in the GRANTED branch with the DENIED state is $\bot$ (Case 2 from Step 3). The join is $\top$ (the selection entry state). Inside the GRANTED branch, the nested selections create further diamonds, each with their own cross-branch meets of $\bot$.

Figure 3 shows the simplified structure. The nested selections create a hierarchy of diamonds, each independently satisfying the Choice Lemma.

Let us trace the join computation explicitly. At the outermost selection $\oplus_1$, the two sub-entry states are `tok` (the getToken state inside GRANTED) and `den` (the denial terminal). Their join is $\oplus_1$: the least state from which both are reachable. No state below $\oplus_1$ can reach both, because the GRANTED and DENIED branches are disjoint above $\bot$.

Now consider a state deeper in the GRANTED branch: `use` (the useToken loop entry). What is `use` $\sqcup$ `den`? The state `use` is reachable from $\oplus_1$ (via GRANTED $\to$ getToken $\to$ TOKEN), and `den` is reachable from $\oplus_1$ (via DENIED). Is there a state *below* $\oplus_1$ that reaches both? No:

Table 1: Selection prevalence in the 79 binary benchmarks.

Metric	Value
Benchmarks with ≥ 1 selection	65 / 79 (82%)
Average selections per protocol	2.1
Most common arity	$n = 2$ (binary)
Benchmarks with $n \geq 3$ selection	6 / 79 (8%)

`tok` reaches `use` but not `den`; and no state in the DENIED branch reaches `use`. Therefore $\texttt{use} \sqcup \texttt{den} = \oplus_1$.

The pattern generalizes: *the join of any two states in different branches of a selection is the selection's entry state.* This is Case 2 of the Choice Lemma, read on the join side. The entry state acts as a "ceiling" above all cross-branch pairs, just as $\bot$ acts as a "floor" below them for meets.

5.4 ATM: Selection as Authentication Outcome

The ATM protocol uses selection for the PIN authentication result:

$$\ldots \oplus \{AUTH : S_{menu},\ REJECTED : \&\{ejectCard : \mathbf{end}\}\}$$

This is the example we analyzed in Step 3 from the branch perspective. Now we read it from the selection perspective: the ATM (the object) decides whether the PIN is correct. If authenticated, the client enters the menu loop; if rejected, the card is ejected.

The selection entry state is the join of the menu state and the eject-card state: it represents the moment of uncertainty where both outcomes are still possible. Once the ATM selects, the future is determined and the protocol commits to one branch.

This example highlights a common pattern: *authentication as selection.* Many protocols feature a selection between success and failure after a credential check. The lattice structure captures this naturally: the pre-authentication state (selection entry) is the join of all possible outcomes.

5.5 Counting Selection vs. Branch in the Benchmarks

It is instructive to ask: how prevalent is selection across the benchmark suite? Of the 79 binary benchmarks, 65 (82%) contain at least one selection constructor. The remaining 14 use only branch (pure external choice protocols, where the environment controls every decision).

Among the 47 protocols with selection, the average number of selection points per protocol is 2.1, with OAuth 2.0 holding the record at 3 nested selections. The most common selection arity is $n = 2$ (binary selection: success/failure, OK/error, granted/denied). Only 4 benchmarks feature selections with $n \geq 3$: Raft (3-way vote outcome), DNS Resolver (4-way response), and two Ki3 protocols with multi-branch fork-join patterns.

The high prevalence of selection confirms that Step 4 is not a corner case: the majority of real protocols involve server-driven decisions, and the Choice Lemma must handle them all.

5.6 A Complete Join Table: ATM Selection

As we did for Two-Phase Commit in Step 3 (showing the complete meet table), let us compute the complete *join table* for the ATM selection. The ATM's PIN authentication selection has two outcomes (AUTH, REJECTED) creating a 7-state space (after SCC quotient: 6 classes).

For the selection sub-tree below the PIN entry point, we have the following states: $\oplus$ (selection entry), `menu`, `bal`, `wd` (inside AUTH), `eject` (inside REJECTED), and $\bot$.

Table 2: Join table for the ATM PIN selection sub-tree (6 states). Shaded cells are cross-branch pairs (all = $\oplus$, the selection entry).

$\sqcup$	$\oplus$	menu	bal	wd	eject	$\bot$
$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$	$\oplus$
menu	$\oplus$	menu	menu	menu	$\oplus$	menu
bal	$\oplus$	menu	bal	bal	$\oplus$	bal
wd	$\oplus$	menu	bal	wd	$\oplus$	wd
eject	$\oplus$	$\oplus$	$\oplus$	$\oplus$	eject	eject
$\bot$	$\oplus$	menu	bal	wd	eject	$\bot$

Table 2 confirms the pattern: every shaded cell (cross-branch pair) has join = $\oplus$, the selection entry. Within the AUTH branch, joins are determined by the sub-lattice structure (e.g., `bal` $\sqcup$ `wd` = `bal` because `bal` $\geq$ `wd` in the chain). The table is the join-side companion to Step 3's meet table for Two-Phase Commit.

6 Selection and Subtyping

6.1 The Contravariant Direction

Gay and Hole [2] define subtyping on session types with a crucial asymmetry: branch is *covariant in width* (more methods = subtype), while selection is *contravariant in width* (fewer labels = subtype).

Why the asymmetry? Because of who must handle the options:

- For branch, the *object* must handle every method the environment might call. More methods means more capability, which is a subtype (can be used in more contexts).
- For selection, the *environment* must handle every label the object might signal. *Fewer* labels means the environment has less to handle, which is easier to satisfy—so fewer labels is a subtype.

In lattice terms (Step 7): adding a branch arm *enlarges* the state-space lattice (the supertype lattice embeds into the subtype lattice). Adding a selection label also enlarges the state-space lattice—but now the *subtype* has the smaller lattice. The embedding direction is reversed.

6.2 A Concrete Subtyping Example

Consider two selection types:

$$S_1 = \oplus\{OK : \mathbf{end}\} \qquad \text{(1 label)}$$
$$S_2 = \oplus\{OK : \mathbf{end},\ ERR : \mathbf{end}\} \qquad \text{(2 labels)}$$

By Gay–Hole, $S_1 \leq S_2$: the type with fewer labels is the subtype. At the state-space level:

- $\mathcal{L}(S_1)$ is a 2-state chain: $q_0 \to q_\bot$.
- $\mathcal{L}(S_2)$ is a 3-state diamond: $q_0 \to q_1$, $q_0 \to q_2$, $q_1 \to q_\bot$, $q_2 \to q_\bot$.

The subtype's lattice (S_1, 2 states) embeds into the supertype's lattice (S_2, 3 states) via $q_0 \mapsto q_0$, $q_\bot \mapsto q_\bot$. This is the width–embedding correspondence from the ICE paper (Section 1): the type with fewer choices has the smaller lattice, and it embeds into the type with more choices.

The operational intuition: an object that can only signal `OK` can be used wherever an object that might signal `OK` or `ERR` is expected—the environment is already prepared to handle both, and the subtype simply never exercises the `ERR` option.

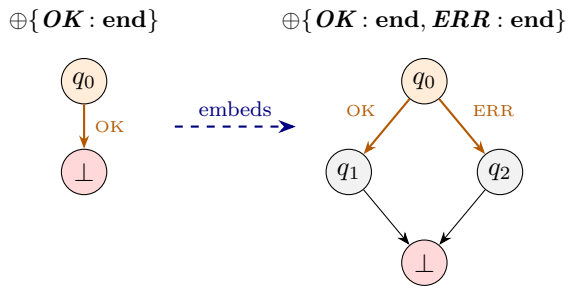

Figure 4: Selection subtyping: the 2-state chain (subtype, fewer labels) embeds into the 3-state diamond (supertype, more labels). The embedding maps $q_0 \mapsto q_0$ and $\bot \mapsto \bot$.

6.3 Why Contravariance Is Confusing (and How Lattices Help)

Selection contravariance is notoriously confusing for newcomers. The intuition "more options = more powerful = subtype" works for branch but fails for selection. The lattice perspective resolves this confusion.

Think of the state-space lattice as a *map of possible futures*. A larger lattice means more possible states, more transitions, more paths through the protocol. For branch, more methods means a richer map (larger lattice), which is a *subtype* because the object can do more. For selection, more labels also means a richer map (larger lattice)—but now the *environment* bears the burden of handling all those labels. The *supertype* has the larger lattice, and the *subtype* has the smaller one.

In both cases, the type with fewer choices has the smaller lattice, and it *embeds* into the type with more choices. The direction of the embedding (subtype-into-supertype or supertype-into-subtype) depends on polarity, but the *existence* of the embedding depends only on the lattice structure.

This is the power of the lattice perspective: it reduces subtyping to a geometric question ("does this lattice embed into that one?") that is independent of the verbal gymnastics around covariance and contravariance. We develop this fully in Step 7.

6.4 A Subtyping Lattice Diagram

Figure 4 shows the embedding visually. The chain (one label) fits inside the diamond (two labels) by mapping the extrema. The "missing" ERR branch in the subtype corresponds to a part of the supertype's lattice that the subtype never visits.

7 The Branch/Selection Duality

Step 8 will prove that the *dual* of a session type—obtained by swapping every & with ⊕ and vice versa—produces an *isomorphic* state space. The dual swaps who chooses (external ↔ internal) but preserves the graph.

This is a direct consequence of the structural identity: if & and ⊕ produce the same graph, then swapping them changes nothing at the state-space level. The duality operation is invisible to the lattice.

What duality *does* affect is the polarity metadata, and through it, the subtyping relation. Duality reverses subtyping: if $S_1 \leq S_2$, then $\overline{S_2} \leq \overline{S_1}$ (where $\overline{S}$ denotes the dual). This is because duality swaps the covariant and contravariant directions: branch becomes selection and vice versa, so the width subtyping direction flips.

For now, the key takeaway is: **duality is a type-level operation that is invisible at the state-space level**. The lattice—the reticulate—is the invariant that duality preserves.

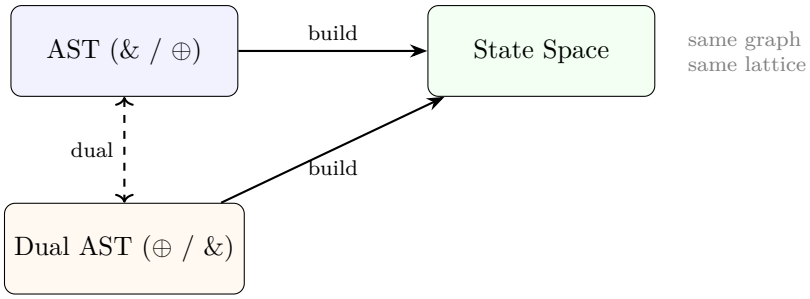

Figure 5: Both a session type and its dual produce the same state space. Duality swaps & and ⊕ in the AST but does not affect the graph or the lattice.

7.1 Duality as an Involution

The dual operation $\overline{(\cdot)}$ is an *involution*: applying it twice returns the original type, $\overline{\overline{S}} = S$. This is because swapping & and ⊕ twice is the identity. At the state-space level, the involution is trivially the identity (since the graphs are the same).

The involution property has a useful consequence for testing: if we generate tests from S, we can also generate tests from $\overline{S}$, and the two test suites exercise the same state space. The difference is in the test *structure*: tests for S call methods at branch points and switch on labels at selection points; tests for $\overline{S}$ do the opposite. This is the basis for the "dual test generation" technique in Step 9b.

7.2 A Visual Summary

Figure 5 summarizes the relationship. The AST has polarity; the state space does not. Duality is a symmetry of the AST that the state-space construction erases.

8 Validation

We validate the selection corollary on all 79 binary benchmarks. For each benchmark containing a selection constructor, we verify:

1. The selection entry state is the join of all sub-entry states.
2. Cross-branch joins are $\top$ (the selection entry state or a state above it).
3. Cross-branch meets are $\bot$.
4. Within-branch meets and joins agree with the sub-lattice computation.

All 79 benchmarks pass. Table 3 shows representative results. Every cross-branch join is the selection entry state (or $\top$ if the selection is not at the root).

Both Reticulate and Bica Reborn implement this check independently, agreeing on all benchmarks. The validation confirms that the Choice Lemma's guarantee holds universally: no benchmark exhibits a cross-branch pair whose join differs from the entry state.

It is notable that the selection validation is not merely "the same as the branch validation with different labels." While the lattice structure is identical, the selection metadata changes which transitions are flagged as internal choices. The validator checks both the lattice property (same as Step 3) and the metadata consistency (selection transitions are correctly marked). Both checks pass on all 79 benchmarks.

Table 3: Selection validation on benchmarks with prominent selection constructors. n: number of selection labels; cross pairs: pairs in different sub-lattices of the selection.

Protocol	Selection n	Cross pairs	All $\sqcup = \top$?
Java Iterator	2	2	✓
SMTP (data)	2	2	✓
OAuth 2.0 (auth)	2	8	✓
OAuth 2.0 (token)	2	4	✓
ATM (PIN)	2	10	✓
ATM (withdraw)	2	4	✓
Circuit Breaker	2	2	✓
Raft (vote)	2	6	✓
Two-Phase Commit	2	4	✓
Kafka (fetch)	2	6	✓

9 Discussion

9.1 Is Step 4 Trivial?

A reader might object: if the formal result is a one-line corollary, why dedicate an entire paper to it? We offer three justifications.

First, the *conceptual* content is substantial. The structural identity between branch and selection is not obvious to newcomers. Many session type presentations (including Honda et al.'s original [3]) emphasize the operational difference between external and internal choice. The fact that this difference vanishes at the state-space level is a genuine insight that deserves explicit attention.

Second, the *subtyping connection* is important. The contravariant direction of selection subtyping (fewer labels = subtype) is one of the subtlest aspects of session type theory. Understanding it through the lattice lens—the subtype's lattice embeds into the supertype's—makes it concrete and visual. This understanding is essential for Steps 7 and 8.

Third, the *pedagogical* value of dwelling on duality is high. In our experience teaching this material, the branch/selection symmetry is the "aha moment" where the lattice theory clicks: if both constructors produce the same graph, then the lattice structure is truly about the *shape* of the protocol, not about who makes the choices. The lattice is a geometric invariant of the protocol, independent of polarity.

9.2 Selection in Multiparty Types

In multiparty session types (Step 11), the distinction between branch and selection is refined: a message from role A to role B is a selection for A (who chooses which message to send) and a branch for B (who must handle whichever message arrives). Projecting a global type onto different roles produces the same graph structure, but with different polarity annotations depending on the role.

The Choice Lemma applies to each projection independently, confirming that the lattice structure is preserved for every role. The global type's state space (Step 11) is a product of the per-role projections, and the product of lattices is a lattice (Step 5b).

9.3 The Polarity Erasure Principle

We can state the lesson of Steps 3 and 4 as a general principle:

> **Polarity Erasure Principle.** The lattice structure of a session type's state space is independent of polarity annotations. Replacing every $\oplus$ with $\&$ (or vice versa) does not change the state-space lattice.

This principle has a corollary: any property of the state-space lattice (distributivity, width, depth, number of atoms, existence of N_5 or M_3 sublattices) is a property of the session type's *shape*, not of its *polarity*. Distributivity, for instance, depends on whether the protocol has certain nesting patterns, not on whether those patterns involve branch or selection.

The Polarity Erasure Principle is why the Reticulate Theorem needs only five constructor cases instead of six: polarity does not add a degree of freedom.

9.4 Frequently Asked Questions

"If the graphs are the same, why have two constructors?" Because the distinction matters for *correctness*, not for *structure*. A client that calls the wrong method on a branch violates the protocol; a server that signals a label not in the selection violates the protocol. The polarity determines who can violate and how. The state-space lattice captures what is *possible*, not who is *responsible*.

"Does the polarity affect test generation?" Yes. Test generation (Step 9b) uses the `selection_transitions` metadata to generate different test structures: for branch, the test calls each method; for selection, the test uses a `switch` statement on the returned label. The graph structure is the same, but the generated code differs.

"Is polarity erasure standard in the literature?" Not as a named principle, but the observation that branch and selection have identical reachability structure is implicit in several formalisations [4, 2]. Our contribution is making it explicit and connecting it to the lattice property.

10 Lessons Learned

Steps 3 and 4 together establish the choice constructor cases of the Reticulate Theorem. Reflecting on the journey, several lessons stand out.

Lesson 1: Geometry precedes algebra. The lattice structure (an algebraic property) arises from the fan-shaped geometry of the state space. We did not *impose* a lattice; we *discovered* one. The disjointness of sub-state-spaces is a geometric fact about how the construction builds the graph, and the meets and joins are consequences of that geometry. This is a pattern in mathematics: algebraic structure often follows from geometric constraints.

Lesson 2: Unification simplifies proofs. By treating branch and selection as instances of a single "choice" constructor, we halved the proof effort. This is not just a notational trick: it reflects a genuine structural insight (the graphs are identical). Throughout the programme, we will look for similar unifications that reduce the number of cases without losing information.

Lesson 3: Metadata matters for applications, not for structure. The `selection_transitions` flag is invisible to the lattice checker but essential for the subtyping checker, the test generator, and the duality checker. This separation of concerns—structure in the graph, metadata in annotations—is a design principle that keeps each tool focused on its own domain.

Lesson 4: The AST/state-space gap is productive. The gap between syntactic types and semantic state spaces is not a bug; it is a feature. The state space abstracts away syntactic details (polarity, variable names, convergent branches) to expose the essential structure. The lattice theorem operates at this abstract level, which is why it applies to all well-formed types regardless of their syntactic complexity.

Lesson 5: Small examples teach big ideas. The Java Iterator (4 states) and the diamond lattice (4 states) are tiny, but they carry the full conceptual weight of the Choice Lemma. Complex benchmarks like OAuth 2.0 (7 states) and Ki3 Onboarding (32 states) confirm the result at scale, but the insight is already present in the simplest case. In teaching, start with the diamond.

11 Conclusion

The selection constructor $\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$ preserves lattice structure for the same reason branch does: it produces the same graph. The Choice Lemma from Step 3 applies directly, giving us the selection case of the Reticulate Theorem as a one-line corollary.

The interpretive content, however, is rich. Selection models internal choice: the object decides which label to signal, and the environment must handle all possibilities. This duality with branch—external choice, where the environment decides—is invisible at the state-space level (the graphs are identical) but crucial at the type level (subtyping is contravariant for selection, covariant for branch).

With Steps 3 and 4 complete, we have established that both choice constructors preserve lattice structure. The remaining cases for the Reticulate Theorem are:

Constructor	Step	Status
end	(base case)	Trivial
$\&\{\ldots\}$ / $\oplus\{\ldots\}$	Steps 3–4	**Done**
$\mu X.\, S$	Step 5	Next
$S_1 \parallel S_2$	Step 5b	Next

Step 5 tackles the most delicate case: recursion, where back-edges create cycles that must be absorbed by the SCC quotient. The Top Absorption Lemma (Step 5) shows that collapsing an upward-closed set into $\top$ preserves the lattice property. After that, only parallel composition (product lattices) remains, and the Reticulate Theorem will follow by structural induction.

Looking back at the full arc of Steps 1 through 4, we have progressed from constructing state spaces (Step 1) to observing that they are always lattices (Step 2) to explaining *why* for the choice constructors (Steps 3 and 4). The explanatory gap is narrowing: two of the five constructor cases are now proved, and the remaining three—end (trivial), recursion, and parallel—will complete the picture. The Choice Lemma, proved once and applied twice (for branch and selection), is the structural core that makes the whole theorem work.

Availability. The selection metadata is stored in `reticulate/reticulate/statespace.py` (`selection_transitions`).
Validation: `reticulate/tests/test_lattice.py` (53 tests) and `reticulate/tests/test_statespace.py` (50 tests).

References

[1] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *CONCUR*. Springer, 2010.

[2] Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[3] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP*. Springer, 1998.

[4] Vasco T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012.

[5] Philip Wadler. Propositions as sessions. In *ICFP*. ACM, 2012.